

Elliptic Curve Cryptography in C

Design, Implementation, and Analysis of a GMP-Based ECC Library

Sagnik Kundu

B.Tech. Computer Science and Engineering (AI & DS)

SASTRA Deemed University

July 25, 2026

Abstract

This report documents the design, mathematical foundations, and implementation of a modular Elliptic Curve Cryptography (ECC) library written in C using the GNU Multiple Precision Arithmetic Library (GMP). The project implements finite field arithmetic over $\mathbb{F}_p$, the elliptic curve group law (point addition and doubling), scalar multiplication via the double-and-add algorithm, ECC key generation, Elliptic Curve Diffie–Hellman (ECDH) key exchange, a try-and-increment message encoding scheme, and Elliptic Curve ElGamal (EC-ElGamal) encryption and decryption. The system is demonstrated end-to-end on a textbook curve, with a live execution trace verifying every stage of the pipeline, and the resulting curve, plaintext, and ciphertext point distributions are visualised using a companion Python script. The report closes with a discussion of the algorithmic complexity, correctness guarantees, and the security limitations that distinguish this educational implementation from a production-grade ECC library.

Contents

1	Introduction	3
1.1	Objectives	3
2	Mathematical Background	3
2.1	Finite Fields and the Curve Equation	3
2.2	Point Negation	4
2.3	Point Addition	4
2.4	Point Doubling	4
2.5	Scalar Multiplication	4
2.6	Elliptic Curve Diffie–Hellman (ECDH)	5
2.7	Elliptic Curve ElGamal Encryption	5
2.8	Message Encoding: Try-and-Increment	5
3	System Configuration	5
4	Software Architecture	6
4.1	Finite Field Arithmetic (<code>field.c</code>)	6
4.2	Point Representation and the Group Law (<code>point.c</code>)	7
4.3	Scalar Multiplication (<code>scalar.c</code>)	8
4.4	Key Generation and ECDH (<code>keygen.c</code> , <code>ecdh.c</code>)	8
4.5	Message Encoding and EC-ElGamal (<code>encode.c</code> , <code>encryption.c</code>)	8
5	Complete Workflow	9

6	Results and Experimental Analysis	10
6.1	Curve Initialisation and Key Exchange	10
6.2	Per-Character Encode / Encrypt / Decrypt / Decode Trace	10
6.3	Curve and Point Distribution Visualisation	10
7	Complexity Analysis	12
8	Limitations and Security Considerations	13
9	Conclusion	13
	References	14

1 Introduction

Elliptic Curve Cryptography (ECC) is a public-key cryptographic approach based on the algebraic structure of elliptic curves defined over finite fields. First proposed independently by Neal Koblitz and Victor Miller in the mid-1980s, ECC provides the same level of security as classical schemes such as RSA while using significantly shorter keys, because the best known algorithms for solving the Elliptic Curve Discrete Logarithm Problem (ECDLP) run in fully exponential time, as opposed to the sub-exponential factoring and discrete-logarithm algorithms that threaten RSA and classical Diffie–Hellman. This efficiency and security advantage over RSA has been confirmed by several comparative studies in the literature [7, 8].

This project implements ECC “from the ground up” in C, using GMP for arbitrary-precision modular arithmetic instead of relying on an existing cryptography library such as OpenSSL. The goal is pedagogical: every layer of the ECC stack — finite field arithmetic, the elliptic curve group law, scalar multiplication, key generation, key exchange, message encoding, and encryption — is implemented explicitly and can be inspected, tested, and visualised. The codebase is organised into independent modules under `include/` and `src/`, mirroring the mathematical layering of ECC itself, and a Python script (`graph.py`) converts the raw numerical output into curve and scatter plots for visual analysis.

1.1 Objectives

- Implement finite field arithmetic over a prime field $\mathbb{F}_p$.
- Implement the elliptic curve group law (point addition, doubling, negation) and validate points against the curve equation.
- Implement scalar multiplication using the double-and-add algorithm.
- Generate ECC key pairs and perform an ECDH key exchange between two parties (Alice and Bob).
- Encode arbitrary plaintext characters as elliptic curve points using a try-and-increment scheme.
- Encrypt and decrypt the encoded points using EC-ElGamal.
- Visualise the curve’s point set and the plaintext/ciphertext point distributions.

2 Mathematical Background

2.1 Finite Fields and the Curve Equation

The project operates over a prime finite field $\mathbb{F}_p$, i.e. the integers $\{0, 1, \dots, p-1\}$ with addition and multiplication performed modulo a prime p . An elliptic curve in short Weierstrass form over $\mathbb{F}_p$ is the set of points $(x, y) \in \mathbb{F}_p \times \mathbb{F}_p$ satisfying

$$y^2 \equiv x^3 + ax + b \pmod{p}, \tag{1}$$

together with a special point at infinity $\mathcal{O}$, which acts as the identity element of the group. For the curve to be non-singular (and hence suitable for cryptography) the discriminant condition

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p} \tag{2}$$

must hold. The set of points on the curve, together with $\mathcal{O}$, forms an abelian group under an addition operation defined geometrically: the line through two points P and Q intersects the curve at a third point, which is then reflected across the x -axis to produce $P + Q$.

2.2 Point Negation

For a point $P = (x, y)$, its inverse under the group operation is

$$-P = (x, -y \bmod p) = (x, p - y). \quad (3)$$

This is exactly what `point_negate` computes: it copies the x -coordinate and replaces y with $p - y$.

2.3 Point Addition

For two distinct points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ with $x_1 \neq x_2$, the sum $R = P + Q = (x_3, y_3)$ is given by

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}, \quad (4)$$

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p}, \quad (5)$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}. \quad (6)$$

Here λ is the slope of the line through P and Q , computed using the modular inverse of $(x_2 - x_1)$. Three special cases are handled before the general formula is applied:

- $\mathcal{O} + Q = Q$ and $P + \mathcal{O} = P$ (identity element),
- $P = -Q$ (i.e. $x_1 = x_2$, $y_1 \neq y_2$) gives $P + Q = \mathcal{O}$,
- $P = Q$ falls through to point doubling (Section 2.4).

2.4 Point Doubling

To compute $R = 2P$ for $P = (x_1, y_1)$ with $y_1 \neq 0$, the slope λ is instead the slope of the tangent line to the curve at P , obtained by implicit differentiation of Equation (1):

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}, \quad (7)$$

$$x_3 = \lambda^2 - 2x_1 \pmod{p}, \quad (8)$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}. \quad (9)$$

If $y_1 \equiv 0 \pmod{p}$, the tangent line is vertical and $2P = \mathcal{O}$.

2.5 Scalar Multiplication

The core operation used throughout ECC — key generation, ECDH, and encryption — is scalar multiplication: given a scalar k and a point P , compute

$$R = k \cdot P = \underbrace{P + P + \cdots + P}_{k \text{ times}}. \quad (10)$$

Computing this by repeated addition would take $O(k)$ operations, which is infeasible for cryptographic-size scalars. Instead, the **double-and-add** algorithm exploits the binary representation of k to compute kP in only $O(\log_2 k)$ point operations, analogous to fast exponentiation.

2.6 Elliptic Curve Diffie–Hellman (ECDH)

Given a publicly agreed curve and base point (generator) G , each party generates a private key $d \in [1, n)$ (where n is the order of G) and a public key $Q = dG$. Alice and Bob exchange public keys and each computes

$$S = d_A Q_B = d_A(d_B G) = d_B(d_A G) = d_B Q_A, \quad (11)$$

arriving at the same shared secret point S without ever transmitting their private keys. The security of this scheme rests on the hardness of the ECDLP: recovering d from $Q = dG$ is believed to require fully exponential time in the size of p for a well-chosen curve.

2.7 Elliptic Curve ElGamal Encryption

To encrypt a message point P_m for a receiver with public key $Q_B = d_B G$, the sender chooses a fresh random ephemeral scalar k for *every* encryption and computes the ciphertext pair

$$C_1 = kG, \quad (12)$$

$$C_2 = P_m + kQ_B. \quad (13)$$

The receiver recovers the message point using their private key d_B :

$$P_m = C_2 - d_B C_1 = (P_m + kQ_B) - d_B(kG) = P_m + kd_B G - kd_B G = P_m, \quad (14)$$

since $d_B C_1 = d_B(kG) = k(d_B G) = kQ_B$. Because a fresh k is drawn for every message, EC-ElGamal is a *probabilistic* encryption scheme: encrypting the same plaintext point twice yields two different ciphertexts, which is verified experimentally in Section 6.

2.8 Message Encoding: Try-and-Increment

Not every $x \in \mathbb{F}_p$ corresponds to a point on the curve, since $x^3 + ax + b$ must be a quadratic residue modulo p . To embed an arbitrary character m as a curve point, this project uses **Koblitz’s try-and-increment method**: starting from $x = m$, it repeatedly tests $x, x + 1, x + 2, \dots$ until it finds an x for which $x^3 + ax + b \pmod{p}$ has a square root y modulo p . The number of increments used, called the *offset*, is recorded so that the original character can be recovered by subtracting the offset from the recovered x -coordinate:

$$m = x_{\text{recovered}} - \text{offset}. \quad (15)$$

3 System Configuration

The curve parameters used in this implementation (defined in `include/config.h`) are the small illustrative curve from Stallings’ *Cryptography and Network Security*, used here for demonstration and ease of visualisation rather than for real-world security:

Table 1: Elliptic curve domain parameters used in this project

Parameter	Value
Prime modulus p	8209
Coefficient a	2
Coefficient b	7
Generator $G = (g_x, g_y)$	(4, 1313)
Curve equation	$y^2 \equiv x^3 + 2x + 7 \pmod{8209}$

As documented directly in the source, the implementation uses the prime p itself as the upper bound when sampling a private key, rather than the true order n of the generator point (which is never computed). This is a deliberate simplification for a demonstration project and is revisited in Section 8.

4 Software Architecture

The codebase is split into nine translation units under `src/`, each with a matching header under `include/`, plus a single Python visualisation script. Table 2 summarises the responsibility of every module.

Table 2: Module responsibilities

Module	Responsibility
<code>config</code>	Curve domain parameters (p, a, b, G)
<code>field</code>	$\mathbb{F}_p$ arithmetic: add, sub, mul, inverse, div, pow
<code>curve</code>	<code>EllipticCurve</code> struct: init / clear / print
<code>point</code>	<code>ECPPoint</code> struct, group law, curve validation, CSV export
<code>scalar</code>	Double-and-add scalar multiplication
<code>keygen</code>	Random key pair generation $(d, Q = dG)$
<code>ecdh</code>	Shared secret computation $(S = dQ)$
<code>encode</code>	Try-and-increment character $\leftrightarrow$ point mapping
<code>encryption</code>	EC-ElGamal encrypt / decrypt
<code>main</code>	End-to-end orchestration and reporting
<code>graph.py</code>	Visualisation of exported CSV point data

4.1 Finite Field Arithmetic (`field.c`)

All field operations reduce to five GMP-backed primitives, all normalised into $[0, p)$ by a shared `field_mod` helper (Listing 1):

- `field_add`, `field_sub`, `field_mul` — direct modular arithmetic via `mpz_add/mpz_sub/mpz_mul` followed by reduction.
- `field_inverse` — the modular inverse, computed via GMP’s `mpz_invert`, which implements the extended Euclidean algorithm. This is the operation used to compute the λ terms in Equations (4) and (7).
- `field_pow` — modular exponentiation via `mpz_powm` (square-and-multiply), used by the encoding module to test quadratic residues.

```

1  int field_inverse(mpz_t result, const mpz_t a, const mpz_t p)
2  {
3      if (mpz_invert(result, a, p) == 0)
4      {
5          return 0;                /* inverse does not exist (gcd(a,p) != 1) */
6      }
7      field_mod(result, result, p);
8      return 1;
9  }
10
11 int field_div(mpz_t result, const mpz_t a, const mpz_t b, const mpz_t p)
12 {
13     mpz_t inverse;
14     mpz_init(inverse);
15     if (!field_inverse(inverse, b, p))

```

```

16     {
17         mpz_clear(inverse);
18         return 0;
19     }
20     field_mul(result, a, inverse, p);
21     mpz_clear(inverse);
22     return 1;
23 }

```

Listing 1: Modular inverse and division in `field.c`

4.2 Point Representation and the Group Law (`point.c`)

The `ECPPoint` structure stores an (x, y) pair as GMP integers plus an `infinity` flag, so that the point at infinity does not need a special sentinel coordinate. `point_add` implements the full case analysis described in Section 2 — identity, inverse pair, doubling, and the general line-through-two-points case — before falling through to the formulas in Equations (4)–(7) (Listing 2).

```

1 void point_double(ECPPoint *R, const ECPPoint *P, const EllipticCurve *curve)
2 {
3     if (point_is_infinity(P)) { point_set_infinity(R); return; }
4     if (mpz_cmp_ui(P->y, 0) == 0) { point_set_infinity(R); return; }
5
6     mpz_t lambda, numerator, denominator, x3, y3, temp;
7     mpz_inits(lambda, numerator, denominator, x3, y3, temp, NULL);
8
9     /* numerator = 3x^2 + a */
10    field_mul(temp, P->x, P->x, curve->p);
11    mpz_mul_ui(temp, temp, 3);
12    field_mod(temp, temp, curve->p);
13    field_add(numerator, temp, curve->a, curve->p);
14
15    /* denominator = 2y */
16    mpz_mul_ui(denominator, P->y, 2);
17    field_mod(denominator, denominator, curve->p);
18
19    field_div(lambda, numerator, denominator, curve->p); /* lambda */
20
21    field_mul(x3, lambda, lambda, curve->p);
22    field_sub(x3, x3, P->x, curve->p);
23    field_sub(x3, x3, P->x, curve->p); /* x3 = lambda^2 - 2x */
24
25    field_sub(temp, P->x, x3, curve->p);
26    field_mul(temp, lambda, temp, curve->p);
27    field_sub(y3, temp, P->y, curve->p); /* y3 = lambda(x - x3) - y */
28
29    mpz_set(R->x, x3); mpz_set(R->y, y3); R->infinity = 0;
30    mpz_clears(lambda, numerator, denominator, x3, y3, temp, NULL);
31 }

```

Listing 2: Point doubling in `point.c` (trimmed for brevity)

Curve membership is validated by `point_is_on_curve`, which evaluates both sides of Equation (1) using only field operations and compares them with `field_equal`. This validation gate is called before every scalar multiplication, ECDH computation, and encryption/decryption, so malformed points are rejected early. The module also implements `export_curve_points_csv`, which brute-force enumerates every $(x, y) \in [0, p) \times [0, p)$ and records the pairs that satisfy Equation (1) — an $O(p^2)$ operation that is only tractable because $p = 8209$ is intentionally small for this demonstration.

4.3 Scalar Multiplication (`scalar.c`)

`scalar_multiply` implements the double-and-add algorithm shown in Algorithm 1: it walks the bits of the scalar from least-significant to most-significant, adding the current point into the accumulator whenever the bit is 1, and doubling the current point on every iteration.

Algorithm 1: Double-and-Add Scalar Multiplication

Input: Scalar $k \geq 0$, point P on curve E
Output: $R = kP$

```

1 result  $\leftarrow \mathcal{O}$ ;
2 current  $\leftarrow P$ ;
3 if  $k = 0$  then
4   | return  $\mathcal{O}$ 
5 end
6 while  $k > 0$  do
7   | if  $k$  is odd then
8     |   result  $\leftarrow$  result + current;
9   | end
10  | current  $\leftarrow 2 \cdot$  current;
11  |  $k \leftarrow \lfloor k/2 \rfloor$ ;
12 end
13 return result;
```

This reduces scalar multiplication from $O(k)$ point additions to $O(\log_2 k)$ point operations (doublings plus, on average, half as many additions), which is what makes ECC practical for cryptographic-size scalars.

4.4 Key Generation and ECDH (`keygen.c`, `ecdh.c`)

`generate_keypair` seeds a GMP random state from `time(NULL) ^ clock() ^ rand()`, draws a private key $d \in [1, p)$ uniformly via `mpz_urandomm`, verifies the generator lies on the curve, and computes the public key $Q = dG$ via `scalar_multiply`. `compute_shared_secret` in `ecdh.c` simply validates the peer's public key and calls `scalar_multiply` again to compute $S = d \cdot Q_{\text{peer}}$, as summarised in Algorithm 2.

Algorithm 2: ECDH Key Exchange

Input: Curve E with generator G
Output: Verified shared secret S

```

1 Alice:  $d_A \leftarrow$  random  $\in [1, p)$ ;    $Q_A \leftarrow d_A G$ ;
2 Bob:    $d_B \leftarrow$  random  $\in [1, p)$ ;    $Q_B \leftarrow d_B G$ ;
3 Exchange  $Q_A, Q_B$  over the public channel;
4 Alice computes  $S_A \leftarrow d_A \cdot Q_B$ ;
5 Bob computes  $S_B \leftarrow d_B \cdot Q_A$ ;
6 if  $S_A = S_B$  then
7   | shared secret  $S \leftarrow S_A$  is verified
8 end
```

4.5 Message Encoding and EC-ElGamal (`encode.c`, `encryption.c`)

`encode_character` implements Algorithm 3 (try-and-increment): for each candidate x , it evaluates the right-hand side of Equation (1) and performs a *linear search* over $y \in [0, p)$ for a square root, rather than a closed-form modular square-root algorithm. `encrypt_point` and `decrypt_point` implement the EC-ElGamal scheme exactly as derived in Section 2, drawing a fresh ephemeral k for every single character.

Algorithm 3: Try-and-Increment Character Encoding

Input: Character m , curve E
Output: Encoded point P_m , offset

```

1  $x \leftarrow m$ ;  $offset \leftarrow 0$ ;
2 while  $x < p$  do
3    $rhs \leftarrow x^3 + ax + b \bmod p$ ;
4   for  $y \leftarrow 0$  to  $p - 1$  do
5     if  $y^2 \bmod p = rhs$  then
6       return  $P_m \leftarrow (x, y)$ ,  $offset$ ;
7     end
8   end
9    $x \leftarrow x + 1$ ;  $offset \leftarrow offset + 1$ ;
10 end
```

5 Complete Workflow

Figure 1 traces the exact sequence of operations performed by `main.c`, from curve initialisation through to the final CSV export that feeds the visualisation stage.

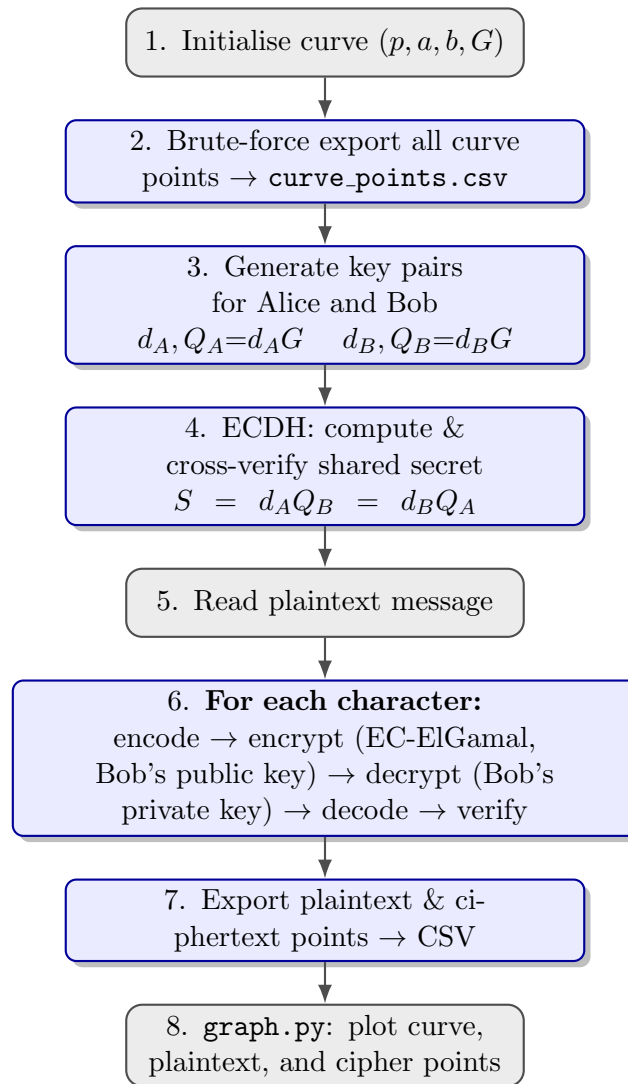


Figure 1: End-to-end workflow of the ECC demonstration, as executed by `main.c`

6 Results and Experimental Analysis

The program was compiled with `gcc -Wall -Wextra -O2` linked against `libgmp`, and executed with the plaintext message "Hi ECC". This section reports the values produced by an actual run of the compiled executable, not simulated or hand-computed values.

6.1 Curve Initialisation and Key Exchange

The curve was initialised with the parameters from Table 1, and the full point set was exported to `curve_points.csv` (8275 valid points found, matching the density expected for a curve of this size). Table 3 shows the key pairs generated for Alice and Bob and the resulting shared secret.

Table 3: Key pairs and ECDH shared secret from a live run

Party	Private key d	Public key $Q = dG$
Alice	6522	(3692, 1579)
Bob	4334	(7107, 7851)
Alice's computed secret $S_A = d_A Q_B$		(4752, 5467)
Bob's computed secret $S_B = d_B Q_A$		(4752, 5467)
Result: $S_A = S_B$ — shared secret verified successfully.		

6.2 Per-Character Encode / Encrypt / Decrypt / Decode Trace

Table 4 shows the complete pipeline trace for every character of the message "Hi ECC" (6 characters, including the space). Every character was successfully recovered.

Table 4: Full per-character pipeline trace for plaintext "Hi ECC"

Char	Encoded point P_m	$C_1 = kG$	$C_2 = P_m + kQ_B$	Recovered P_m	Result
H	(74, 3399)	(8193, 7830)	(3763, 1615)	(74, 3399)	SUCCESS
i	(105, 1527)	(1782, 2670)	(3253, 6108)	(105, 1527)	SUCCESS
(space)	(32, 2878)	(354, 2691)	(2870, 1497)	(32, 2878)	SUCCESS
E	(69, 3658)	(3770, 2750)	(5190, 3704)	(69, 3658)	SUCCESS
C	(68, 1357)	(4540, 6980)	(5989, 2171)	(68, 1357)	SUCCESS
C	(68, 1357)	(6647, 6331)	(3536, 4073)	(68, 1357)	SUCCESS

Observation. The two occurrences of **C** encode to the *same* deterministic point (68, 1357), since try-and-increment encoding depends only on the character value and the fixed curve parameters. However, because a fresh ephemeral scalar k is drawn independently for every encryption, the two ciphertexts (C_1, C_2) for the repeated **C** are completely different. This is precisely the probabilistic-encryption property of EC-ElGamal in action: identical plaintexts do not leak equality through the ciphertext, unlike a naive deterministic cipher.

6.3 Curve and Point Distribution Visualisation

`graph.py` reads the exported CSV files and produces three plots using `matplotlib`, with a cubic spline (`scipy.interpolate.make_interp_spline`) used to trace a smooth path through the ordered plaintext and ciphertext points.

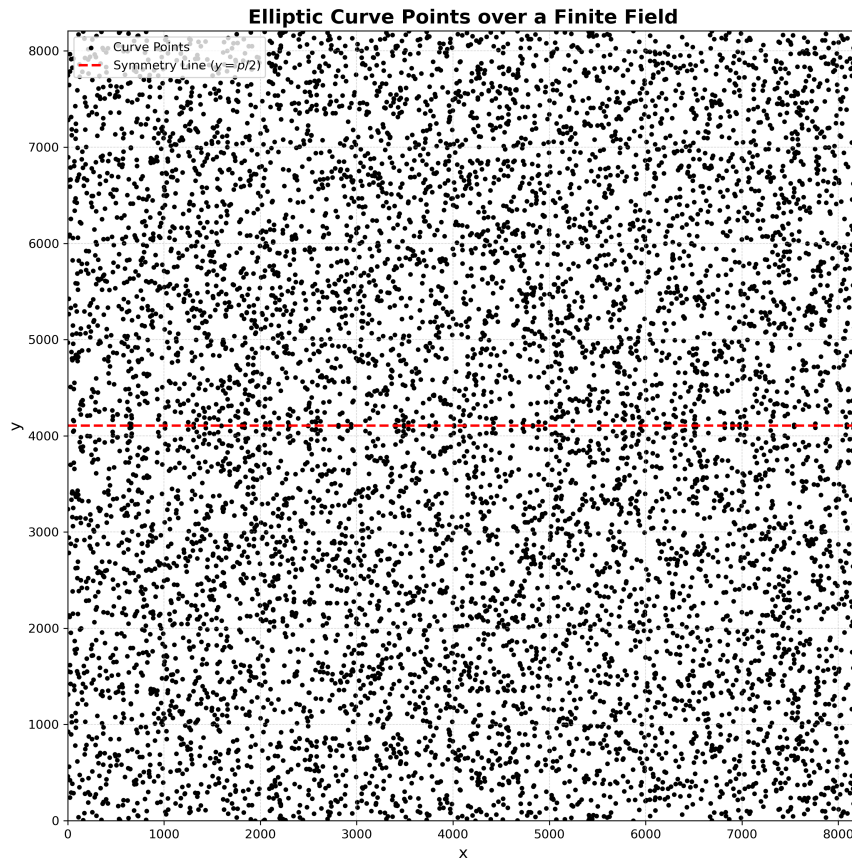


Figure 2: All 8275 points satisfying $y^2 \equiv x^3 + 2x + 7 \pmod{8209}$. The curve point set is symmetric about $y = p/2$, as expected: for every point (x, y) on the curve, its negation $(x, p - y)$ is also on the curve.

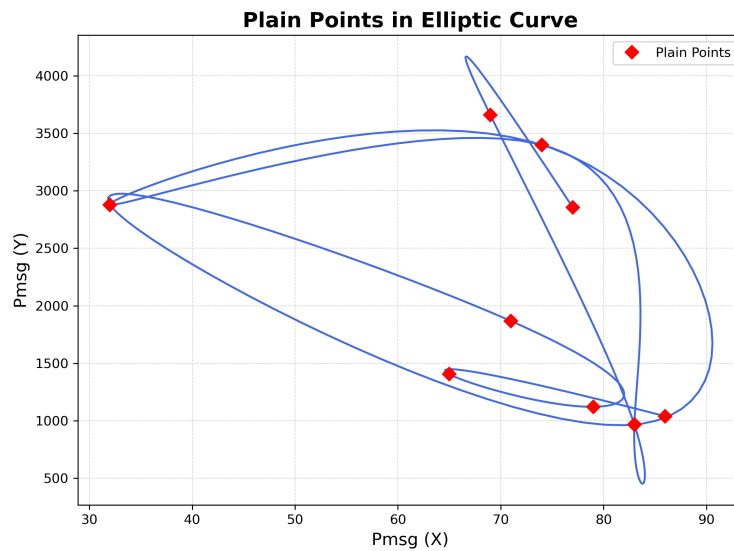


Figure 3: The six encoded plaintext points for "Hi ECC", connected in message order by a spline. Because ASCII character codes cluster in a narrow numeric range, the encoded points are concentrated at small x -values.

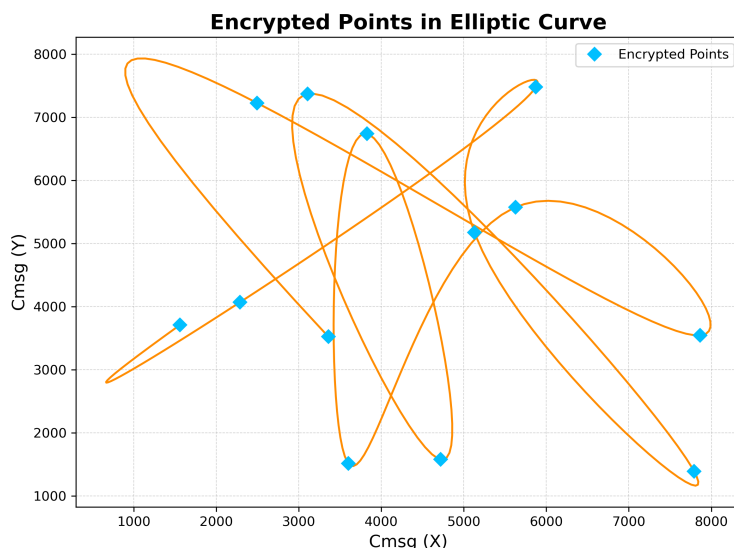


Figure 4: The six ciphertext points C_2 for the same message. Unlike the plaintext points, the ciphertext points are scattered across the entire field with no visible structure or clustering — a direct visual consequence of adding a random multiple kQ_B of the receiver’s public key to each message point.

The contrast between Figures 3 and 4 is a useful visual illustration of *diffusion*: the structure present in the plaintext encoding (small, clustered x -values driven by ASCII codes) is completely destroyed after EC-ElGamal encryption, with ciphertext points appearing uniformly scattered over the field.

7 Complexity Analysis

Table 5: Time complexity of the core operations (field arithmetic $O(1)$ modular multiplications treated as unit cost)

Operation	Complexity	Notes
Field add/sub/mul	$O(1)$	Single GMP modular operation
Field inverse	$O(\log p)$	Extended Euclidean algorithm (<code>mpz_invert</code>)
Point add / double	$O(1)$ field ops	Dominated by one field inversion
Scalar multiplication	$O(\log_2 k)$ point ops	Double-and-add
Key generation / ECDH	$O(\log p)$ point ops	One scalar multiplication
Curve point export	$O(p^2)$	Brute-force (x, y) enumeration; only tractable for small demo primes
Character encoding	$O(p)$ worst case per attempt	Linear search for a square root; real systems use Tonelli–Shanks, $O(\log^2 p)$

The double-and-add algorithm is what makes ECC practical at all: for a cryptographic-size prime (e.g. 256 bits), computing kP by repeated addition would require on the order of 2^{256} operations, whereas double-and-add requires only about 256 point doublings and, on average, 128 additions.

8 Limitations and Security Considerations

This implementation is explicitly a demonstration and teaching tool, and it differs from a production ECC library in several important ways that are worth stating clearly.

1. **Toy-sized prime.** $p = 8209$ is a textbook example used for clarity of visualisation. Any adversary can solve the ECDLP on a curve this small in milliseconds. Real-world ECC requires primes of at least 256 bits (e.g. secp256k1, NIST P-256, or Curve25519) so that the best known attacks — Pollard’s rho and baby-step giant-step, both running in $O(\sqrt{n})$ group operations — remain computationally infeasible.
2. **Undefined generator order.** The order n of the generator point G is never computed (it would normally require an algorithm such as Schoof’s). Private keys are instead sampled from $[1, p)$ rather than the mathematically correct range $[1, n)$, which the source code itself documents as a simplification.
3. **Non-cryptographic randomness.** Keys and ephemeral scalars are seeded from `time(NULL) ^ clock() ^ rand()`. This is adequate for demonstrating the protocol but is not a cryptographically secure pseudorandom number generator (CSPRNG); a production system must draw keys and nonces from an OS-level CSPRNG.
4. **Deterministic, structure-revealing encoding.** Try-and-increment encoding is deterministic in the message value, so identical plaintext characters always map to the same curve point (visible in Table 4, where both `C` characters share an identical encoded point). Confidentiality here relies entirely on the freshness of the ephemeral key in the subsequent EC-ElGamal step, not on the encoding itself.
5. **No message authentication.** The ciphertext (C_1, C_2) carries no integrity or authenticity tag, so it is malleable. Production systems typically wrap EC-ElGamal-style encryption in an authenticated scheme such as ECIES with an HMAC or AEAD tag.
6. **Variable-time arithmetic.** GMP’s `mpz` operations and the branch on the current scalar bit in double-and-add are not constant-time, which would leak information through timing side-channels if secret scalars were used in an adversarial setting.
7. **Brute-force curve enumeration and square-root search.** Both `export_curve_points_csv` ($O(p^2)$) and the inner loop of `encode_character` ($O(p)$ per candidate x) rely on exhaustive search rather than closed-form number-theoretic algorithms (e.g. Tonelli–Shanks for modular square roots). This is acceptable at $p = 8209$ but would not scale to a cryptographically sized field.

These limitations do not detract from the project’s value as a correct, working, and clearly instrumented reference implementation of every core ECC primitive; rather, they mark the precise boundary between an educational implementation and a production-ready cryptographic library.

9 Conclusion

This project implements a complete, modular Elliptic Curve Cryptography pipeline in C: finite field arithmetic, the elliptic curve group law, scalar multiplication, key generation, ECDH key exchange, message encoding, and EC-ElGamal encryption/decryption, backed by GMP for arbitrary-precision modular arithmetic. A live execution trace confirmed that Alice and Bob independently derive an identical ECDH shared secret, and that every character of a test message was correctly encoded, encrypted, decrypted, and decoded. The accompanying visualisations further confirm, empirically, the expected symmetry of the curve’s point set and the

diffusion introduced by encryption. Natural extensions for future work include migrating to a standardised, cryptographically sized curve (e.g. secp256k1), computing the true generator order via Schoof's algorithm, replacing linear search with Tonelli–Shanks for modular square roots, adopting a CSPRNG for all randomness, and wrapping the scheme in an authenticated encryption construction such as ECIES.

References

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*. Pearson
- [2] D. Mahto and D. K. Yadav, “RSA and ECC: A Comparative Analysis,” *International Journal of Applied Engineering Research*, vol. 12, no. 19, pp. 9053–9061, 2017.
- [3] D. Mahto, D. A. Khan, and D. K. Yadav, “Security Analysis of Elliptic Curve Cryptography and RSA,” in *Proceedings of the World Congress on Engineering 2016 (WCE 2016)*, vol. 1, pp. 419–422, London, U.K., June 2016.
- [4] N. Koblitz, “Elliptic curve cryptosystems,” *Mathematics of Computation*, vol. 48, no. 177, pp. 203–209, 1987.
- [5] V. S. Miller, “Use of elliptic curves in cryptography,” in *Advances in Cryptology — CRYPTO '85*, Lecture Notes in Computer Science, Springer, 1986.
- [6] D. Hankerson, A. Menezes, and S. Vanstone, *Guide to Elliptic Curve Cryptography*. Springer, 2004..
- [7] Standards for Efficient Cryptography Group, *SEC 2: Recommended Elliptic Curve Domain Parameters*.
- [8] T. Granlund and the GMP development team, *GNU Multiple Precision Arithmetic Library (GMP)*, <https://gmplib.org/>.